

Quiz — 1.4.2 Data Structures — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (8 marks — 1 each) 1. **B** — stack is LIFO. 2. **B** — record = named fields, possibly different types. 3. **B** — static size is fixed at declaration. 4. **B** — circular queue: $\text{rear} = (\text{rear} + 1) \text{ MOD size}$. 5. **C** — in-order (L, root, R) of a BST gives ascending order. 6. **B** — collision = two keys hash to the same index. 7. **C** — linked lists have no random access; must traverse from the head. 8. **C** — enqueue at rear, dequeue at front.

Section B

9. [5] (1 mark each) a = **Tuple** b = **Stack** c = **Array** d = **Linked list** e = **Hash table**

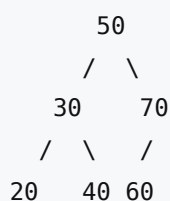
10. [4] Stack trace (top listed first): - push(5) → [5] - push(9) → [9, 5] - push(2) → [2, 9, 5] - pop() → returns **2** → [9, 5] - push(7) → [7, 9, 5] - pop() → returns **7** → [9, 5]

Values popped (in order): **2, then 7**. Final stack (top → bottom): **9, 5**. Mark: correct order of pushes onto top (1); first pop returns 2 (1); second pop returns 7 (1); correct final contents 9,5 (1).

11. [3] Queue trace (front listed first): - enqueue(A) → [A] - enqueue(B) → [A, B] - dequeue() → returns **A** → [B] - enqueue(C) → [B, C] - enqueue(D) → [B, C, D] - dequeue() → returns **B** → [C, D]

Values dequeued (in order): **A, then B**. Queue contents (front → rear): **C, D**. Mark: dequeue returns A then B in that order (1); FIFO applied at correct ends (1); final contents C, D (1).

12. [3] BST after inserting 50, 30, 70, 20, 40, 60:



- 50 is the root.
- $30 < 50 \rightarrow$ left of 50; $70 > 50 \rightarrow$ right of 50.
- $20 < 50, < 30 \rightarrow$ left of 30; $40 < 50, > 30 \rightarrow$ right of 30.
- $60 > 50, < 70 \rightarrow$ left of 70. Mark: 50 root with 30 left / 70 right (1); 20 left and 40 right of 30 (1); 60 as left child of 70 (1).

13. [2] Any two of: - Size: static is fixed at declaration; dynamic grows/shrinks at run time. (1) - Memory: static uses a contiguous block reserved up front; dynamic is allocated from the heap as needed. (1) - Trade-off: static gives fast/simple access but may waste or run out of memory; dynamic is flexible but adds pointer/management overhead. (1) (Max 2 marks.)

Section C

- 14. [5]** (a) **[2]** A **stack** (1). The Back button needs the **most recently visited** page first — Last In, First Out — so pushing each new page and popping to go back returns pages in the correct reverse order (1).
- (b) **[2]** A **queue** (1). A video buffer must release frames in the **order they arrived** — First In, First Out — so enqueueing incoming frames and dequeueing for playback preserves the correct sequence (1).
- (c) **[1]** A dynamic implementation lets the structure **grow or shrink at run time**, so it adapts to an unknown/varying number of items without wasting memory or overflowing a fixed size (1).

Total: 8 + 17 + 5 = 30 marks.